

Analyzing the Scalability of Managed Language Applications with Speedup Stacks



VRIJE
UNIVERSITEIT
BRUSSEL

Jennifer B. Sartor

Kristof Du Bois

Stijn Eyerman

Lieven Eeckhout



■ Multicore

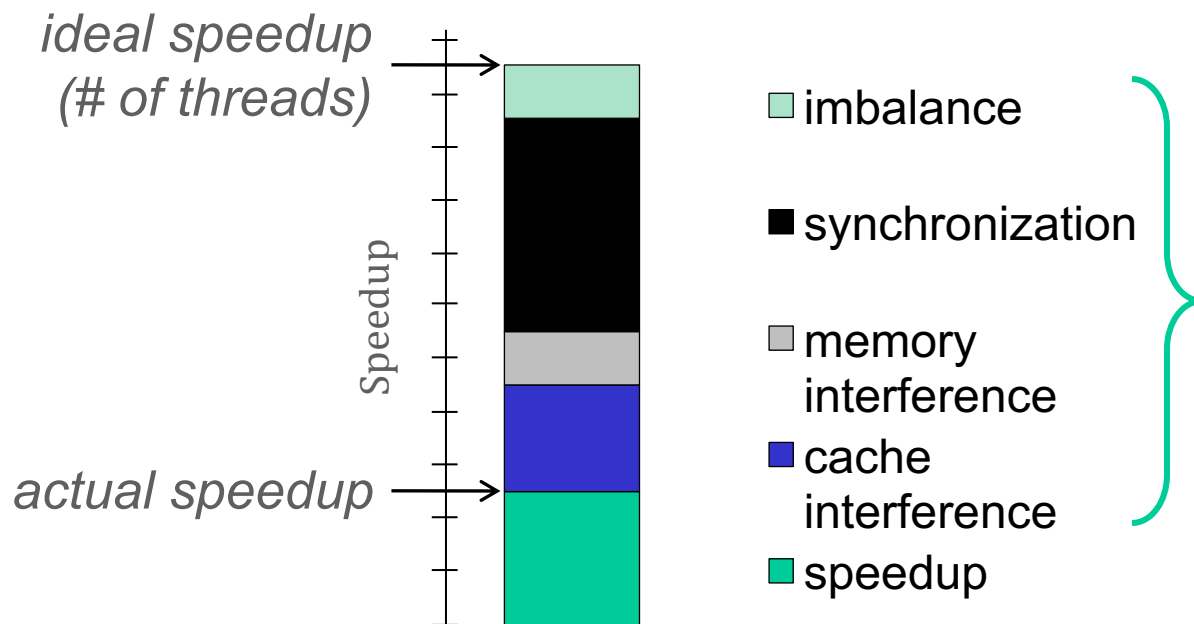
■ Managed languages

- Service threads

➤ Speedup Stack

- Bar graph that explains causes of sublinear speedup
- Ideal speedup of multi-threaded execution over single-threaded versus actual speedup

Original Speedup Stacks



Speedup delimiters:

This factor is responsible for reducing speedup by this amount from the ideal speedup

or

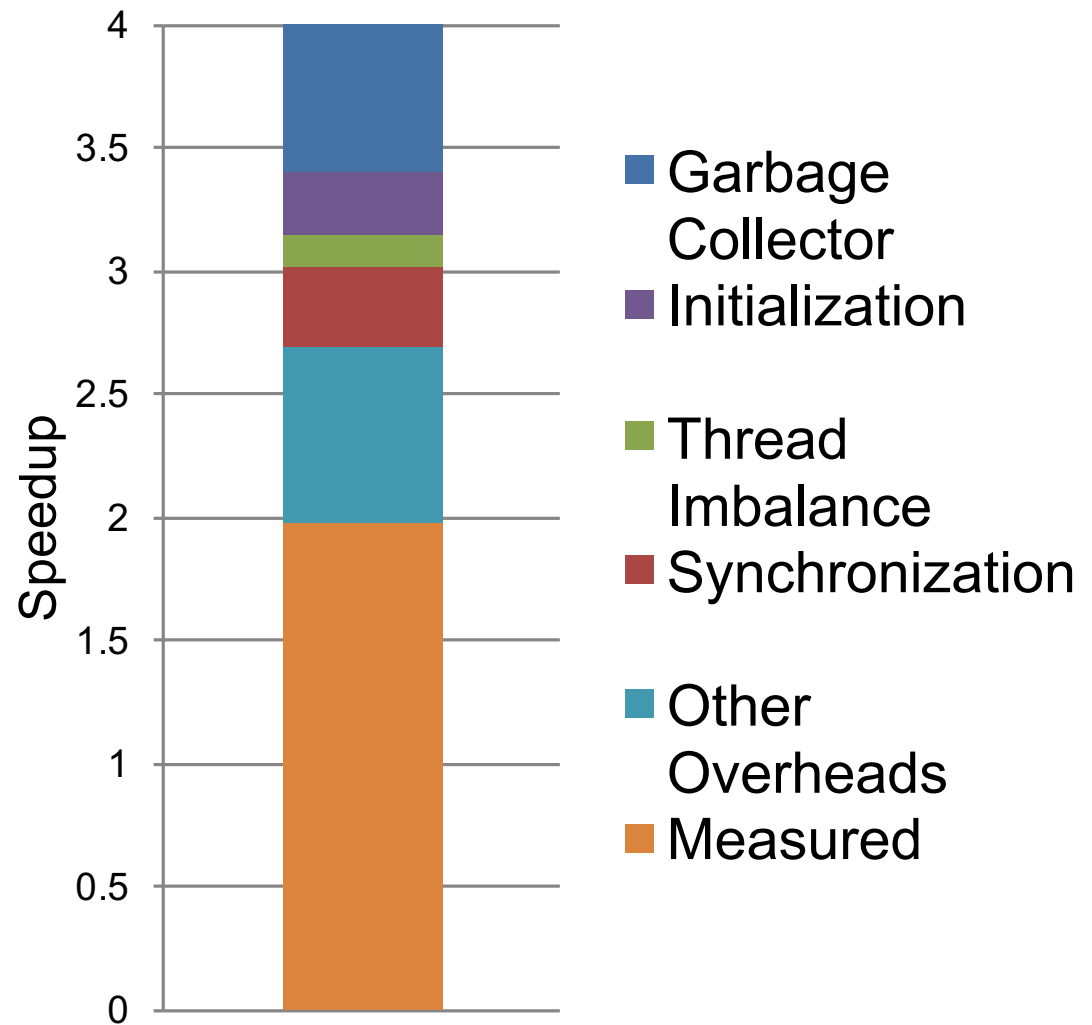
If completely removed, gives indication of how much speedup could improve

■ Scalability delimiters

- Work imbalance
 - Spinning
 - Yielding
 - Last-level cache and memory interference
 - ◆ **Positive**
 - ◆ **Negative**
-
- ✗ No managed components
 - ✗ Dedicated hardware support

Our Contribution

- **Managed service threads**
- **On native hardware**



■ Scalability delimiters

- **Garbage collector** 10%*
- **Managed runtime initialization**
- Synchronization
- Thread imbalance
- Other overheads
 - ◆ Parallelization overhead
 - ◆ Shared hardware resource interference

■ On native hardware

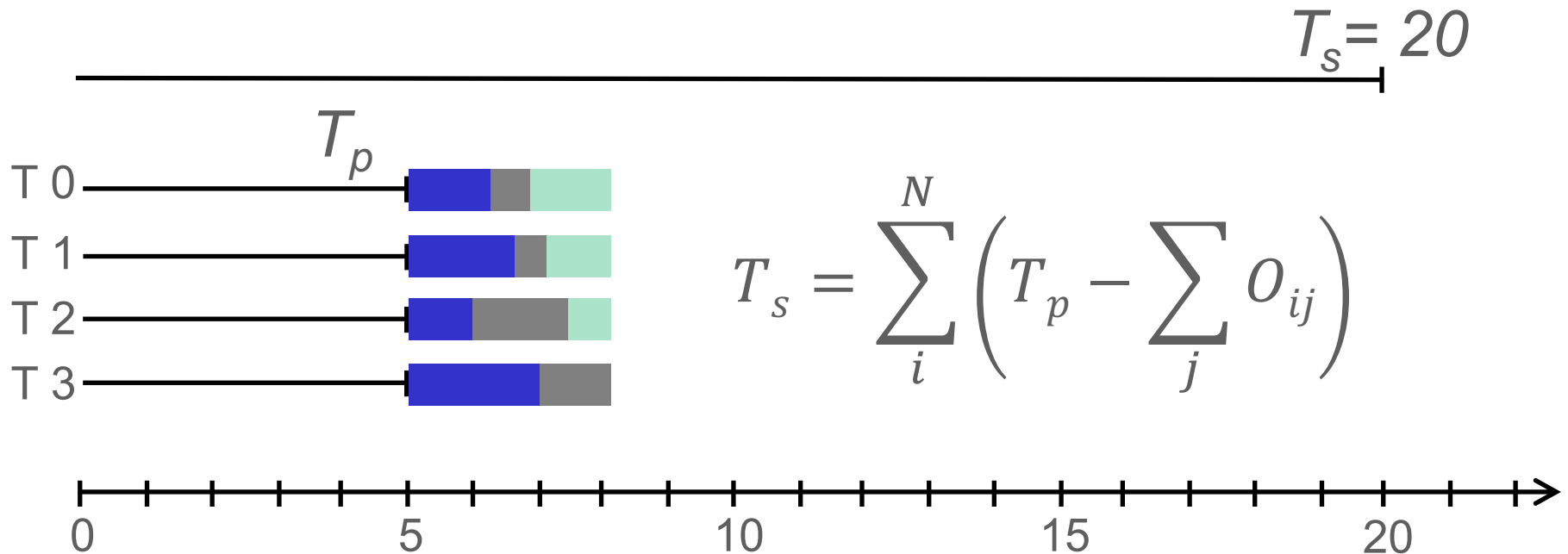
- Linux kernel modules
- < 1% overhead on average

*T. Cao, S. M. Blackburn, T. Gao, and K. S. McKinley, "The yin and yang of power and performance for asymmetric hardware and managed software," ISCA, 2012

$$\text{Speedup} = \frac{\text{time}_{\text{single-thread}}}{\text{time}_{\text{multi-thread}}}$$

$$S = \frac{T_s}{T_p}$$

- Ideal speedup = # of threads (N)

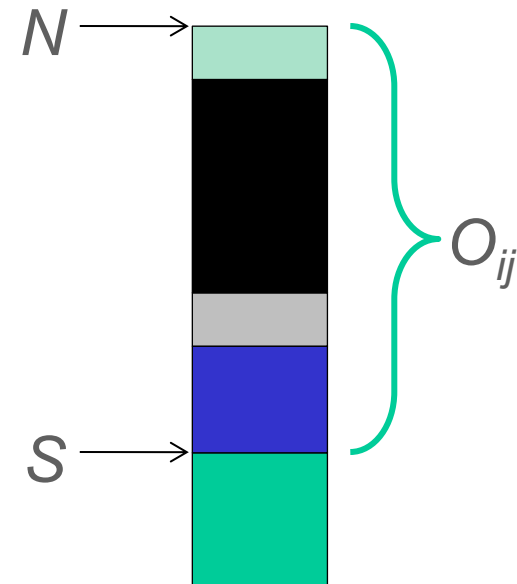


$$\text{Speedup} = \frac{\text{time}_{\text{single-thread}}}{\text{time}_{\text{multi-thread}}} \quad S = \frac{T_s}{T_p}$$

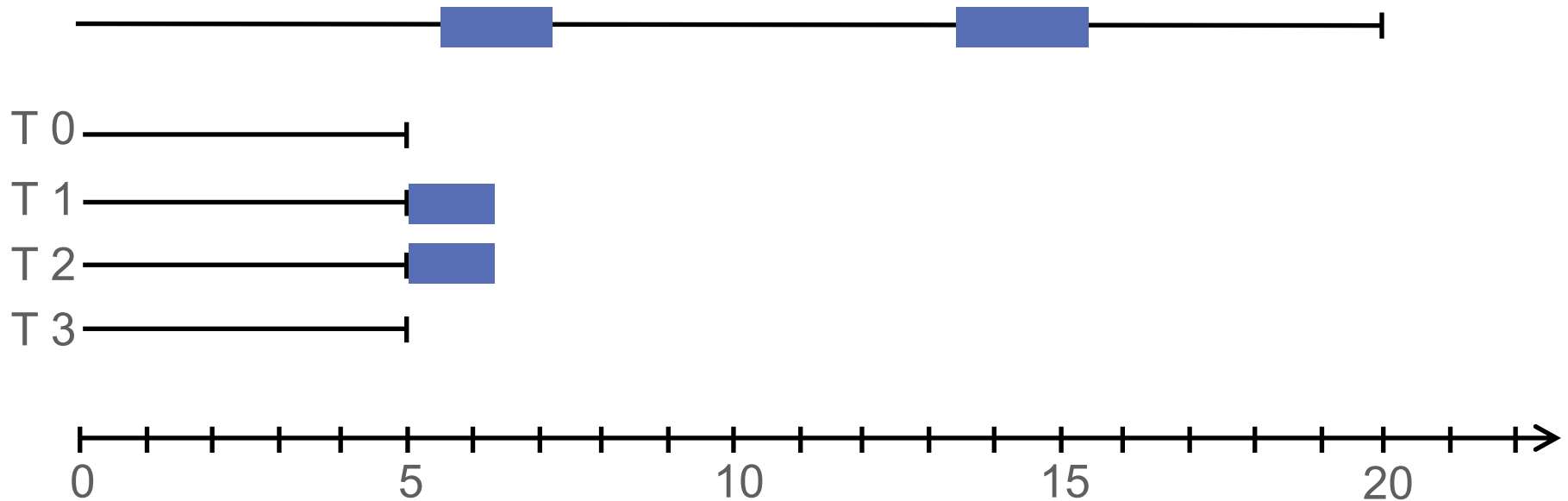
- Ideal speedup = # of threads (N)

$$T_s = \sum_i^N \left(T_p - \sum_j O_{ij} \right)$$

$$\frac{T_s}{T_p} = S = N - \frac{\sum_i^N (\sum_j O_{ij})}{T_p}$$



- When application paused
- In original speedup stacks: part of yielding



- When application paused
- In original speedup stacks: part of yielding

$$S = N \left[\frac{N \times T_{GC, MT} - T_{GC, ST}}{T_p} \right] - \sum_i^N \frac{\sum_j O1_{ij}}{T_p}$$

- If GC were perfectly scalable, component would be 0

-

- Java virtual machine initialization, compilation, shutdown
- Application threads not yet spawned, or paused

$$\begin{aligned}
 &= N \left[\frac{N \times T_{GC, MT} - T_{GC, ST}}{T_p} \right] \\
 &\quad - \left[\frac{N \times T_{init, MT} - T_{init, ST}}{T_p} \right] - \sum_i^N \frac{\sum_j O2_{ij}}{T_p}
 \end{aligned}$$

- If initialization were perfectly scalable, component would be 0

■ Synchronization

- When threads wait on each other
- Measure wait time inside *futex* syscall

■ Thread Imbalance

- When thread executes longer than other threads
- Measure wait time inside *exit* syscall

■ Other Overhead

- Parallelization overhead
- Hardware interference
- Estimated

Managed Speedup Stack

Measured
speedup

$$S = N$$

threads

Garbage
collector

$$\frac{N \times T_{GC, MT} - T_{GC, ST}}{T_p}$$

Initialization

Thread
imbalance

$$\frac{N \times T_{init, MT} - T_{init, ST}}{T_p}$$

Synchronization

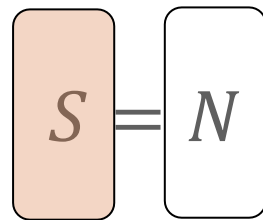
Other
overheads

$$\frac{\sum_i^N Sync_i}{T_p}$$

$$\frac{\sum_i^N Exit_i}{T_p}$$

$$\sum_i^N \frac{\sum_j^N O4_{ij}}{T_p}$$

Managed Speedup Stack

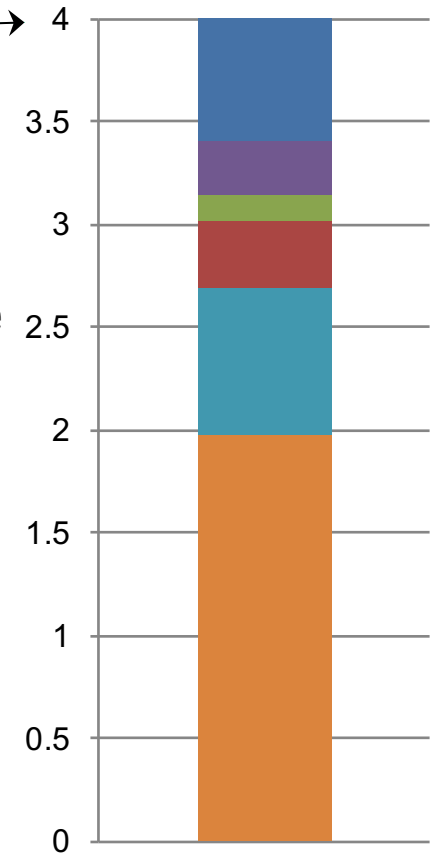


$$-\frac{N \times T_{GC, MT} - T_{GC, ST}}{T_p}$$

$$-\frac{N \times T_{init, MT} - T_{init, ST}}{T_p}$$

$$-\frac{\sum_i^N Sync_i}{T_p} - \frac{\sum_i^N Exit_i}{T_p} - \sum_i^N \frac{\sum_j O4_{ij}}{T_p}$$

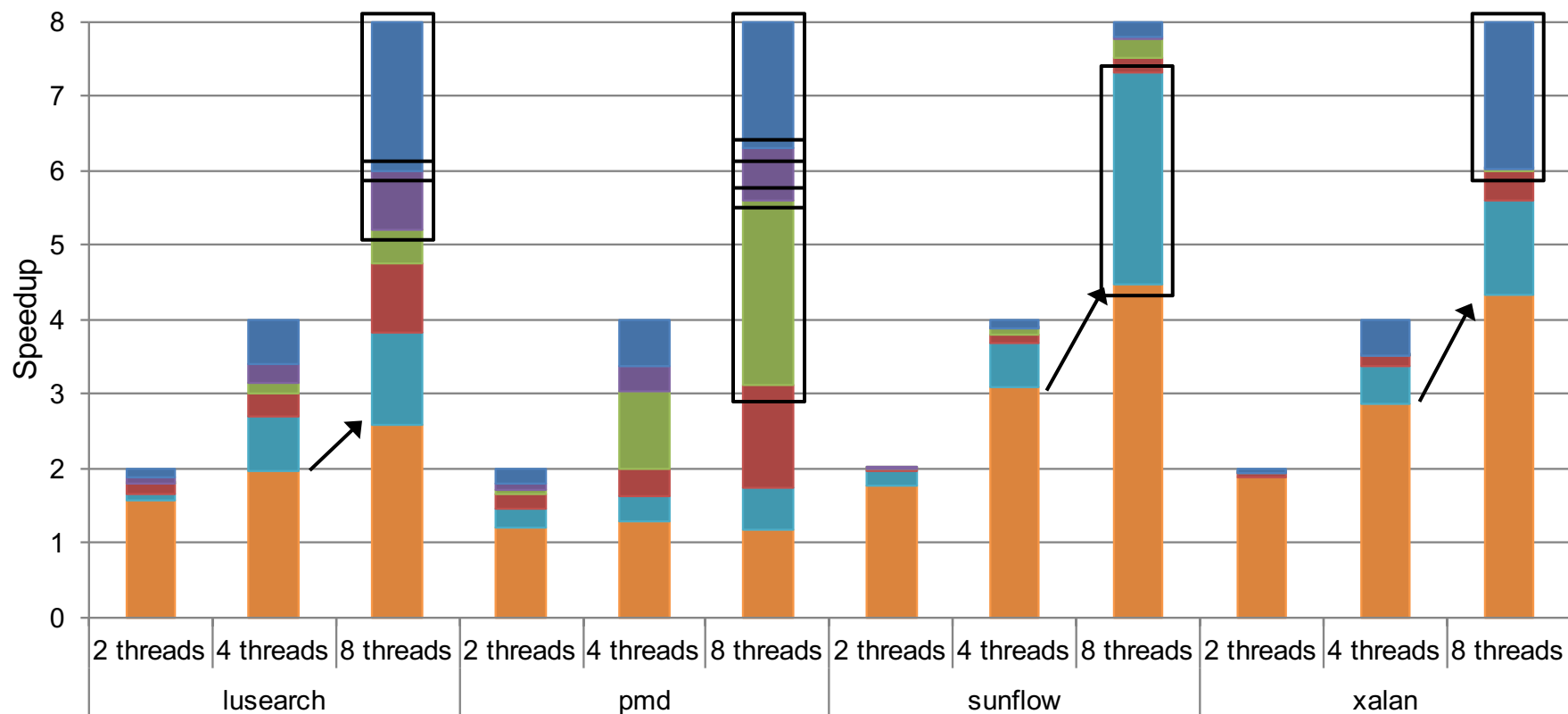
- GC
- Initialize
- Imbalance
- Sync.
- Other
- Measured



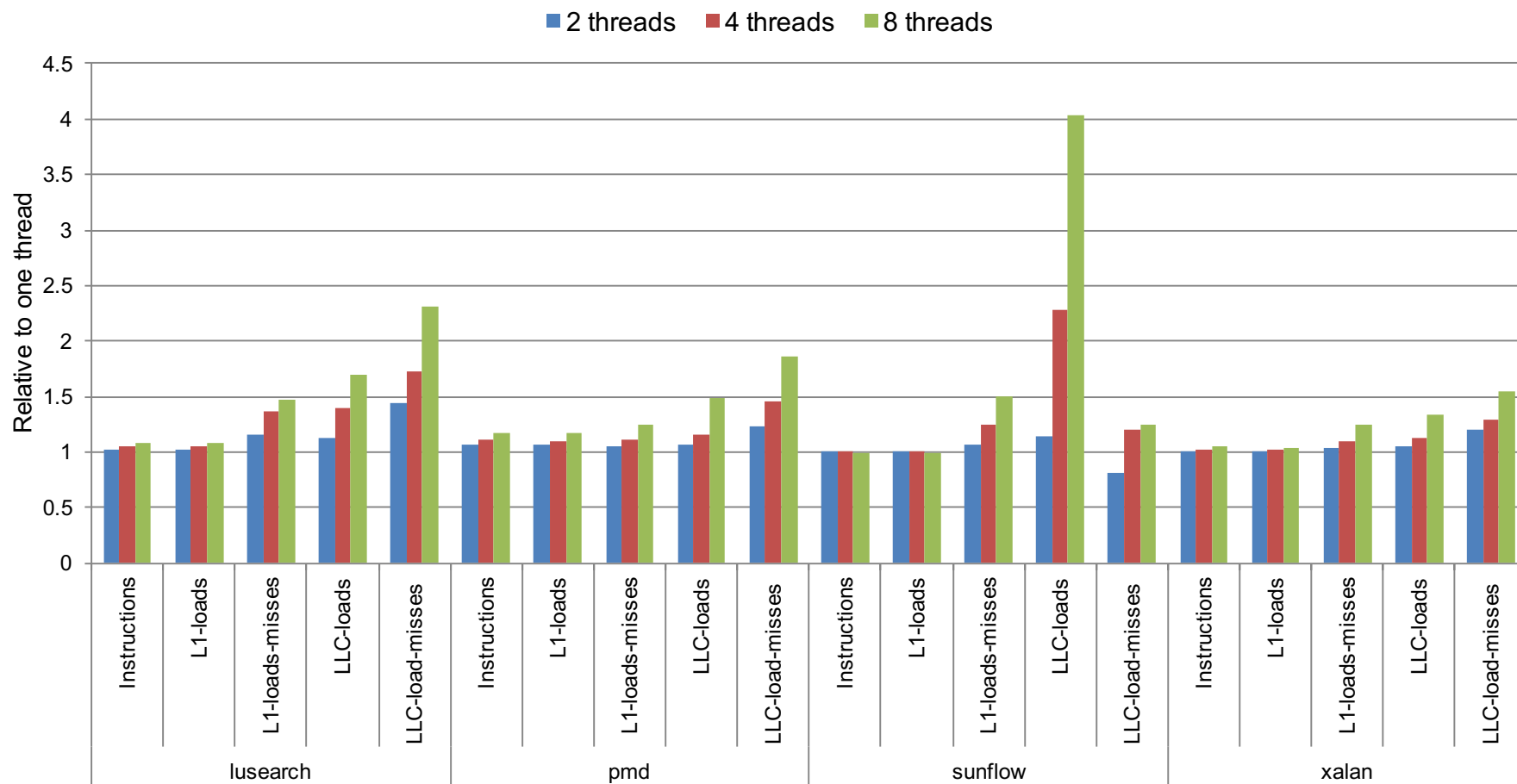
- Java applications from DaCapo 2009 suite
- Jikes Research Virtual Machine 3.1.2
- Garbage collector
 - 2 threads
 - 13th iteration for stable behavior
 - Heap size based on minimum with stop-the-world (STW) collector
 - STW generational Immix and concurrent collectors
- Intel Xeon E5, 8 cores per socket, 20MB LLC
- 3.2.37 Linux kernel

Speedup Stacks with STW GC

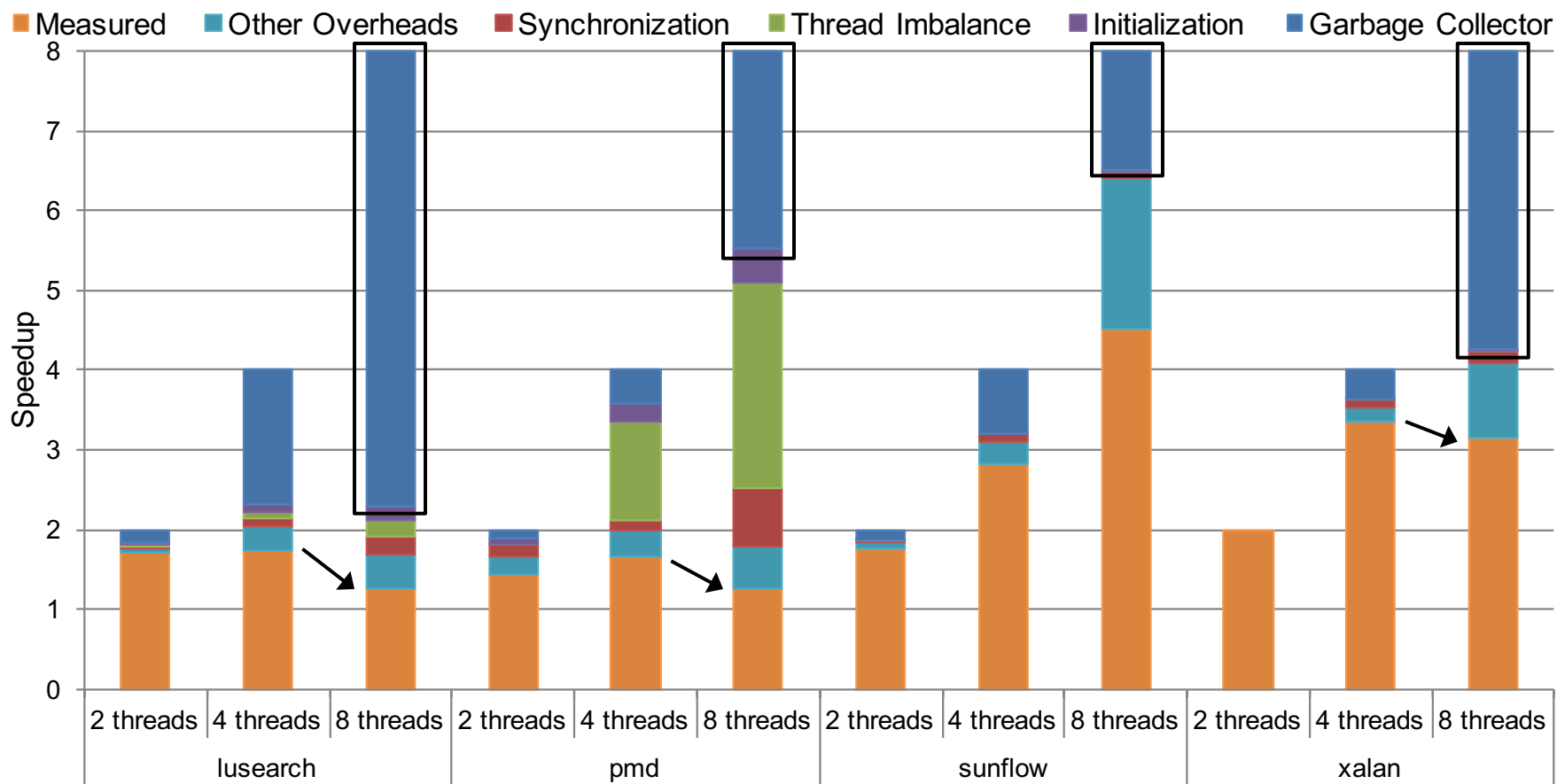
■ Measured
 ■ Other Overheads
 ■ Synchronization
 ■ Thread Imbalance
 ■ Initialization
 ■ Garbage Collector



Performance Counters for STW GC

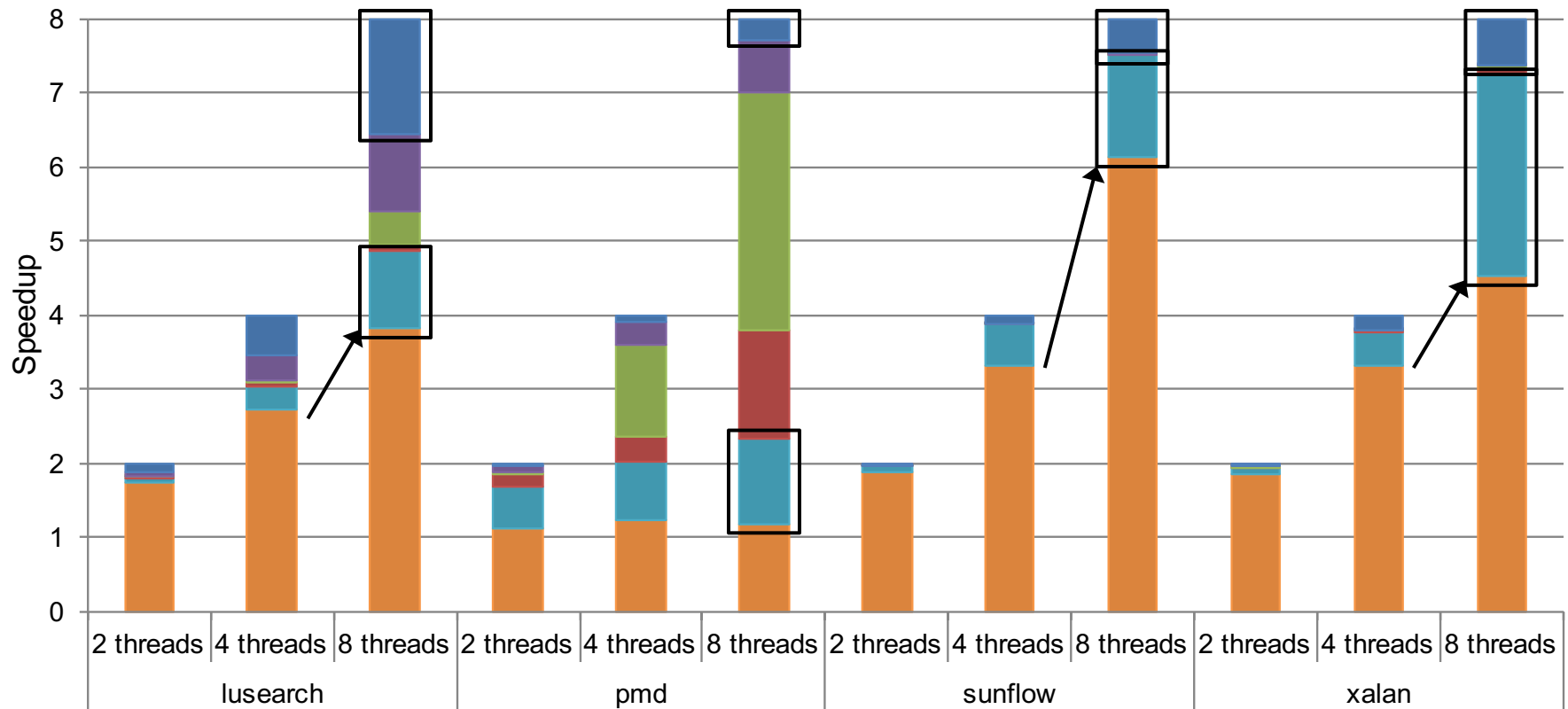


Concurrent GC, Same Heap Size

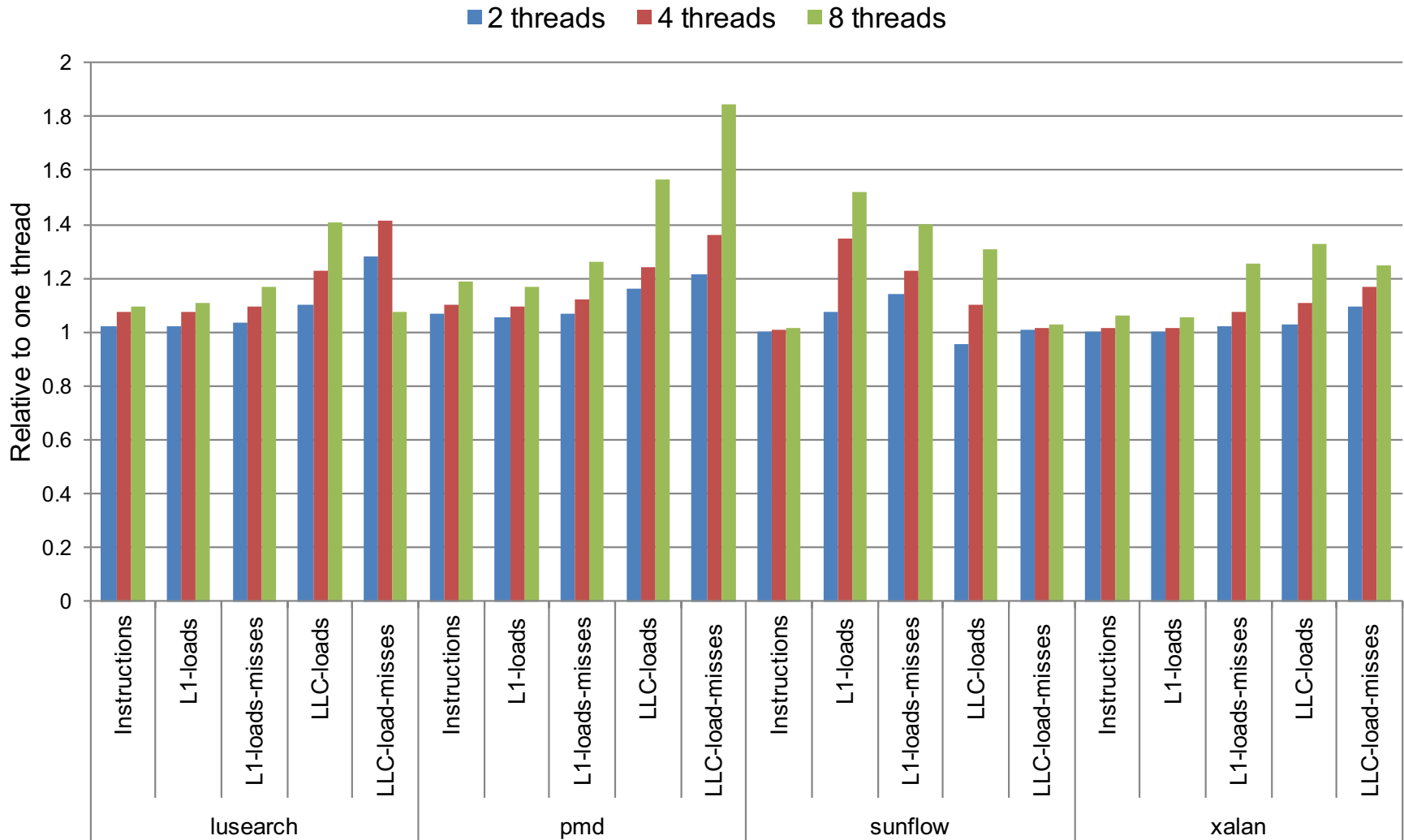


Concurrent GC, Large Heap

■ Measured
 ■ Other Overheads
 ■ Synchronization
 ■ Thread Imbalance
 ■ Initialization
 ■ Garbage Collector

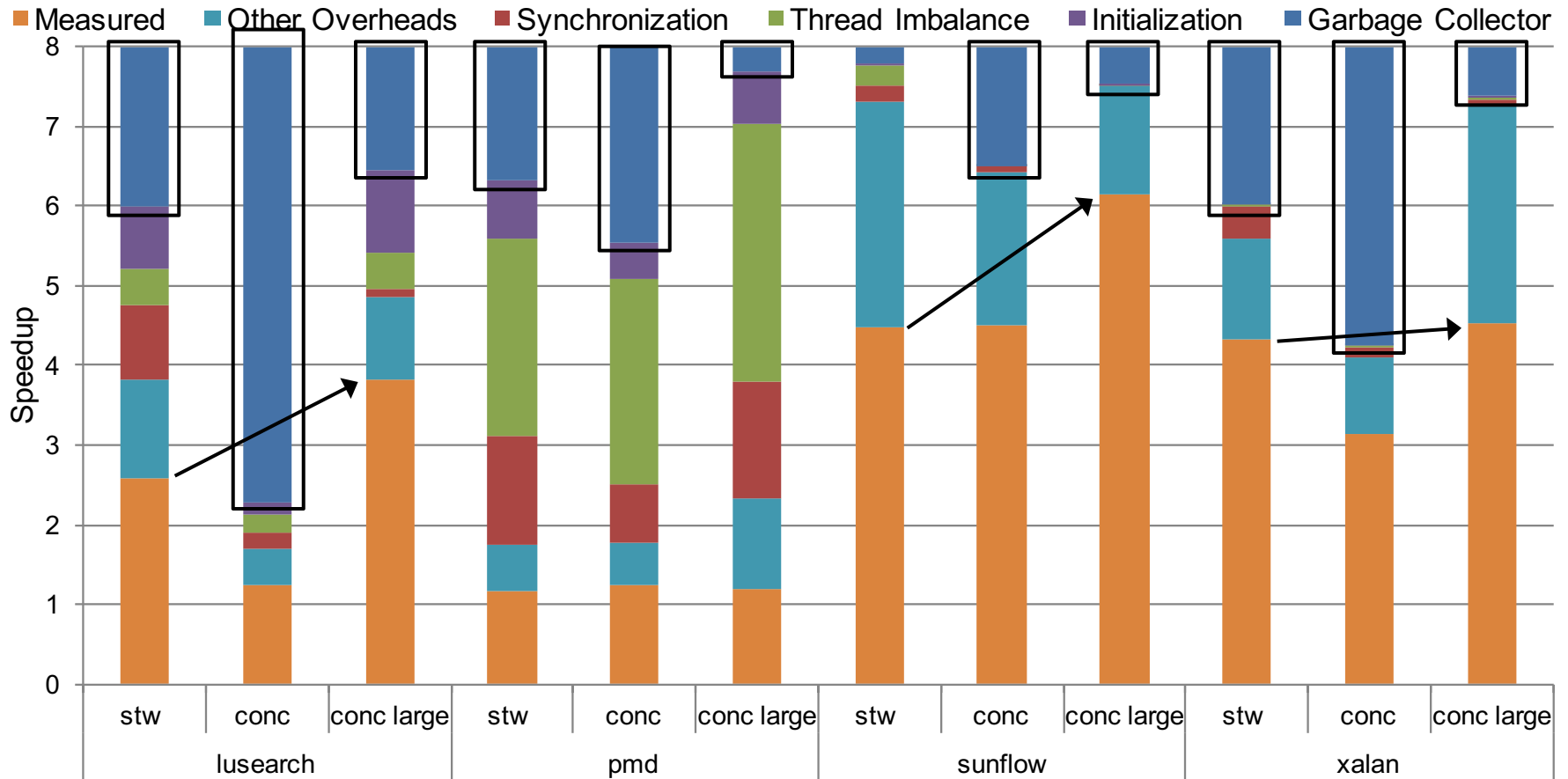


Perf Cntrs, Concurrent GC, Large Heap



Comparison Across Collectors

8 threads



■ Commerical

- Intel VTune Amplifier XE
- Sun Studio Performance Analyzer
- Rogue Wave/Acumem ThreadSpotter
- PGPROF

■ IBM WAIT

■ Criticality stacks & Bottle graphs

- **None quantify gross scalability bottlenecks, most don't analyze service threads**

- Visualize scalability bottlenecks
- Show relative contributions of components
 - Garbage collector
 - Managed runtime initialization
- On native hardware at low overhead
- Show where to focus optimization: application or service threads

